

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	16
REFERÊNCIAS.....	17
PALAVRAS-CHAVE.....	18

EMSE

O QUE VEM POR AÍ?

Nesta aula conheceremos o framework LangChain, que foi desenvolvido para construir fluxos de trabalho com Large Language Models (LLMs). Veremos alguns exemplos de código, como: prompts eficazes, encadeamento de diferentes etapas de processamento de texto e utilização da memória de conversação para manter o contexto ao longo das interações.



HANDS ON

Nesta aula prática, conheceremos o LangChain por meio de exemplos, iniciando por uma requisição simples na API da OpenAI. Vamos lá?

EXEMPLO

SAIBA MAIS

Continuando a seção do Hands On, notem que a chamada é bem simples, nós estamos importando a biblioteca OpenAI e fazendo uma requisição simples que deve retornar uma string.

```
from langchain_openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()

api_key = os.getenv("OPENAI_API_KEY")

llm = OpenAI(api_key=api_key)

response = llm.invoke("Explique o conceito de aprendizado por reforço.")

print(response)
```

No LangChain, temos as **Output Parsers**, que são responsáveis por estruturar as respostas de LLMs em formatos padronizados e úteis, como JSON, tabelas ou strings formatadas. Isso permite que as saídas possam ser utilizadas em etapas subsequentes da aplicação.

Aqui está um exemplo básico de uso de Output Parsers para garantir que uma resposta seja retornada como um JSON válido:

```
from langchain_openai import OpenAI
from langchain.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field
from dotenv import load_dotenv
import os
```

```

load_dotenv()

api_key = os.getenv("OPENAI_API_KEY")

llm = OpenAI(api_key=api_key)

class Pessoa(BaseModel):
    nome: str = Field(..., description="O nome da pessoa")
    idade: int = Field(..., description="A idade da pessoa")
    profissao: str = Field(..., description="A profissão da pessoa")

parser = PydanticOutputParser(pydantic_object=Pessoa)

prompt = """
Formate a seguinte resposta em JSON com os campos em lowercase e sem
caracteres especiais:
"Nome: Maria, Idade: 30, Profissão: Engenheira"
Responda no seguinte formato: {"nome": "valor", "idade": valor, "profissao": "valor"}
"""

response = llm.invoke(prompt)

print("Resposta original do LLM:", response)

try:
    parsed_response = parser.parse(response)
    print("Resposta formatada:", parsed_response)
except Exception as e:
    print("Erro ao usar o parser:", e)

```

O LangChain nos permite adicionar **memória** nas interações, criando sistemas que "lembram" informações ao longo de uma conversa. Essa memória pode ser usada para criar assistentes personalizados, que mantêm o histórico de diálogos para fornecer respostas mais contextualizadas.

```

from langchain_openai import OpenAI
from langchain.memory import ConversationBufferMemory
from dotenv import load_dotenv
import os

load_dotenv()

api_key = os.getenv("OPENAI_API_KEY")

llm = OpenAI(api_key=api_key)

memory = ConversationBufferMemory()

conversation = [
    {"role": "user", "content": "Qual é o meu nome?"},
    {"role": "assistant", "content": "Desculpe, não sei seu nome. Como você se chama?"}
]

memory.add_messages(conversation)

prompt = """
Você agora tem o contexto da conversa que já ocorreu. Continue a conversa como se estivesse ciente da interação anterior.
"""

response = llm.invoke(prompt + str(memory.buffer))

print("Resposta do modelo:", response)
print("Memória da conversa:", memory.buffer)

```

Ele também nos permite **encadear prompts**, onde a saída de um LLM pode ser usada como entrada para outro. Isso é útil em fluxos de trabalho complexos, como diálogos em múltiplas etapas ou decisões baseadas em várias consultas. Aqui está um exemplo de como encadear duas etapas:

```
from langchain_openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()

api_key = os.getenv("OPENAI_API_KEY")

llm = OpenAI(api_key=api_key)

def primeira_etapa(input_text):
    prompt = f"Analise o seguinte texto: {input_text}"
    return llm.invoke(prompt)

def segunda_etapa(analisado):
    prompt = f"Agora resuma a análise: {analisado}"
    return llm.invoke(prompt)

def executar_cadeia(input_text):
    analise = primeira_etapa(input_text)
    print("Análise:", analise)

    resumo = segunda_etapa(analise)
    return resumo

resultado = executar_cadeia("Os impactos da IA na sociedade moderna.")

print("Resultado final:", resultado)
```

Essa foi uma rápida introdução do que podemos fazer utilizando esta biblioteca, e nas próximas aulas avançaremos mais neste assunto.

LangChain

No cenário atual de desenvolvimento de software, a integração de modelos de linguagem natural (Language Models ou LLMs) tornou-se essencial para a criação de aplicações inteligentes e interativas. No entanto, combinar diferentes modelos e serviços em um fluxo de trabalho coeso pode ser um desafio, e é nesse contexto que o LangChain se destaca. Veremos, nesta seção, uma introdução sobre essa biblioteca em diferentes aspectos, desde o acesso a LLMs até a construção de aplicações conversacionais com dados próprios.

O **LangChain** é uma biblioteca projetada para ajudar pessoas desenvolvedoras a construir "cadeias" que conectam diferentes etapas ou módulos baseados em LLMs. O objetivo principal é fornecer uma estrutura na qual podemos combinar esses modelos com outras fontes de informação, interações com APIs, bancos de dados etc., possibilitando a integração de diversas etapas, como criação de prompts, integração com bancos de dados, manipulação de documentos e muito mais.

É amplamente utilizado para construir assistentes virtuais, sistemas de recuperação de informações, ferramentas de análise de texto e aplicações de IA que aprendem com o tempo, graças à sua capacidade de armazenar e reutilizar o contexto de interações.

Ele permite que os(as) desenvolvedores(as) integrem facilmente modelos de linguagem como o **GPT-4**, **LLaMA2**, ou outros LLMs, a suas aplicações. Para isso, a biblioteca fornece conectores que facilitam o envio de prompts e o recebimento de respostas. Na figura 1 há uma demonstração da estrutura básica desta biblioteca:

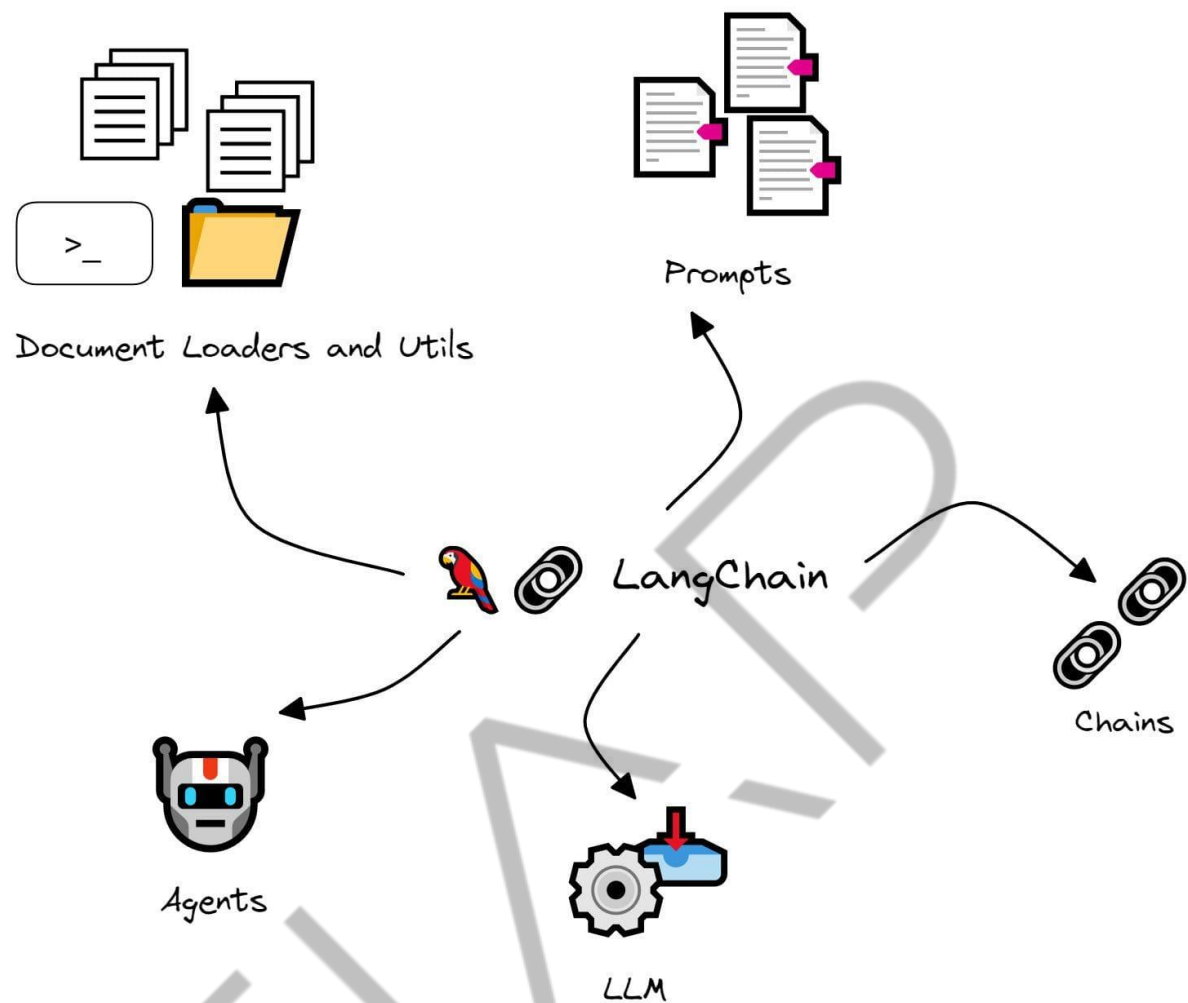


Figura 1 - Estrutura LangChain
Fonte: Daniil Berezhanskiy (2024)

Vamos conhecer cada um dos pontos desta imagem nos tópicos abaixo:

- **Document Loaders and Utils:** essa parte do LangChain se refere aos carregadores de documentos e utilitários. Eles são responsáveis por ler, processar e carregar documentos de diversas fontes (arquivos de texto, PDFs, bancos de dados etc.) para que possam ser utilizados nas operações de processamento de linguagem natural.
- **Prompts:** prompts são instruções ou perguntas fornecidas ao modelo de linguagem para gerar respostas ou realizar uma tarefa específica. Eles são fundamentais para direcionar o comportamento do modelo de linguagem.
- **Chains (ou cadeias):** elas representam uma sequência de etapas ou operações que são realizadas em conjunto para alcançar um objetivo

específico. Embora o uso de um único LLM possa ser suficiente para tarefas mais simples, o LangChain fornece uma interface padrão e algumas implementações comumente usadas para encadear LLMs para aplicações mais complexas entre si ou com outros módulos especializados.

- **LLM (Large Language Models):** LLMs são modelos de linguagem de grande escala, como GPT-4, BERT e outros, que são utilizados para realizar várias tarefas de processamento de linguagem natural, como tradução, sumarização e geração de texto.
- **Agents:** eles são componentes que agem de forma autônoma para realizar tarefas específicas baseadas em entradas e em modelos de linguagem, podendo tomar decisões e realizar ações com base em regras ou aprendizagem.

Para ficar mais claro como esta biblioteca funciona, vejamos um exemplo de como acessar o GPT-4:

```
from langchain_openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()

api_key = os.getenv("OPENAI_API_KEY")

llm = OpenAI(api_key=api_key)

response = llm.invoke("Explique o conceito de aprendizado por reforço.")

print(response)
```

Analisando este trecho de código nós temos:

```
from langchain_openai import OpenAI
from dotenv import load_dotenv
```

```
import os
```

langchain_openai.OpenAI: este módulo é utilizado para se conectar à API da OpenAI e enviar prompts para o modelo GPT (ou outro LLM da OpenAI). Ele facilita o acesso e interação com o modelo.

dotenv.load_dotenv: a biblioteca dotenv carrega as variáveis de ambiente a partir de um arquivo .env. Isso é útil para armazenar chaves de API ou outras informações sensíveis de maneira segura.

os: a biblioteca os é usada para acessar variáveis de ambiente do sistema, como a chave da API armazenada no arquivo .env.

```
load_dotenv()
```

O método **load_dotenv()** carrega as variáveis de ambiente a partir de um arquivo .env que deve estar na mesma pasta ou em um caminho acessível. Esse arquivo .env contém informações sensíveis, como chaves de API, que não devem ser expostas diretamente no código.

```
api_key = os.getenv("OPENAI_API_KEY")
```

O método **os.getenv("OPENAI_API_KEY")** busca o valor da variável de ambiente **OPENAI_API_KEY**. Essa variável contém a chave da API da OpenAI, necessária para fazer chamadas à API e acessar os modelos.

```
llm = OpenAI(api_key=api_key)
```

OpenAI(api_key=api_key): aqui, o objeto LLM (abreviação de "Large Language Model") é inicializado com a chave da API que foi recuperada. Este objeto será usado para enviar solicitações para o modelo GPT-4 (ou outro LLM) da OpenAI.

```
response = llm.invoke("Explique o conceito de aprendizado por reforço.")
```

llm.invoke(): este método envia um **prompt** ao modelo de linguagem. Neste caso, o prompt é: **"Explique o conceito de aprendizado por reforço."**. O modelo

processa essa solicitação e retorna uma resposta que é armazenada na variável `response`.

```
print(response)
```

`print(response)`: exibe a resposta gerada pelo modelo GPT-4, que provavelmente será uma explicação sobre o conceito de **aprendizado por reforço**.

A seguir, temos um print demonstrando o resultado da execução deste código:



```

exemplo1.py > ...
1 from langchain_openai import OpenAI
2 from dotenv import load_dotenv
3 import os
4
5 load_dotenv()
6
7 api_key = os.getenv("OPENAI_API_KEY")
8
9 llm = OpenAI(api_key=api_key)
10
11 response = llm.invoke("Explique o conceito de aprendizado por reforço.")
12
13 print(response)

```

PROBLEMS OUTPUT TERMINAL PORTS POLYGLOT NOTEBOOK DEBUG CONSOLE CODE REFERENCE LOG Python - ia-conference

```

(base) thiagoadriano@Thiagos-MBP ia-conference % /opt/anaconda3/bin/python /Users/thiagoadriano/Documents/FIAP/MLENG/LangChain/aula1/exemp
lo1.py

O aprendizado por reforço é um tipo de aprendizado em que um agente (pode ser um ser humano ou uma máquina) aprende a tomar decisões baseado nas consequências de suas ações. Ele se
baseia no princípio da tentativa e erro, em que o agente realiza uma ação, recebe um feedback (reforço) sobre essa ação e, a partir desse feedback, aprende a escolher a ação mais
adequada para uma determinada situação.

O reforço pode ser positivo (quando uma ação é recompensada) ou negativo (quando uma ação é punida). O agente aprende a maximizar as recompensas e minimizar as punições, a fim de a
lcançar seus objetivos.

O aprendizado por reforço é frequentemente utilizado em ambientes dinâmicos e complexos, onde não é possível prever todas as situações possíveis e, portanto, é necessário aprender
a partir da experiência. Ele é amplamente utilizado em áreas como inteligência artificial, robótica, economia, psicologia e jogos.

Uma das principais características do aprendizado
(base) thiagoadriano@Thiagos-MBP ia-conference %

```

Figura 2 - Resultado da execução do código
Fonte: elaborado pelo autor (2024)

Antes de finalizarmos a nossa aula, vamos instalar em nosso ambiente o “Ollama”, uma ferramenta que nos auxilia na integração de alguns modelos de LLMs, como: llama3.1, gemma2 e mistral-nemo, no nosso ambiente local.



Get up and running with large language models.

Run Llama 3.1, Phi 3, Mistral, Gemma 2, and other models. Customize and create your own.

Download ↓

Available for macOS, Linux,
and Windows (preview)

Figura 3 - Ollama ferramenta
Fonte: Ollama [s.d.]

Caso tenha interesse em saber os modelos que podemos trabalhar utilizando-o, acesse a [biblioteca do Ollama](#), além de checar a imagem 4 a seguir, que exemplifica alguns dos modelos que podemos utilizar.

Featured

llama3.1

Llama 3.1 is a new state-of-the-art model from Meta available in 8B, 70B and 405B parameter sizes.

Tools
8B
70B

↓ 2.3M Pulls
🏷️ 95 Tags
🕒 Updated 4 weeks ago

gemma2

Google Gemma 2 is a high-performing and efficient model by now available in three sizes: 2B, 9B, and 27B.

2B
9B
27B

↓ 948.5K Pulls
🏷️ 94 Tags
🕒 Updated 3 weeks ago

mistral-nemo

A state-of-the-art 12B model with 128k context length, built by Mistral AI in collaboration with NVIDIA.

Tools
12B

↓ 112.1K Pulls
🏷️ 17 Tags
🕒 Updated 4 weeks ago

mistral-large

Mistral Large 2 is Mistral's new flagship model that is significantly more capable in code generation, mathematics, and reasoning with 128k context window and support for dozens of languages.

Figura 4 - Lista de modelos Ollama
Fonte: Ollama [s.d.]

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, conhecemos o framework LangChain, que foi desenvolvido para construir fluxos de trabalho com Large Language Models (LLMs). Vimos alguns exemplos de código, como a criação de prompts eficazes, o encadeamento de diferentes etapas de processamento de texto e o uso da memória de conversação para manter o contexto ao longo das interações.



REFERÊNCIAS

CONHECENDO o LangChain. **dev.to**. 2024. Disponível em: <https://dev.to/programadriano/conhecendo-o-langchain-egk>. Acesso em: 06 set. 2024.

O QUE é o LangChain? **AWS**. 2024. Disponível em: <https://aws.amazon.com/pt/what-is/langchain/>. Acesso em: 06 set. 2024.

THE LARGEST community building the future of LLM apps. **langchain**. 2024. Disponível em: <https://www.langchain.com/langchain>. Acesso em: 06 set. 2024.

PALAVRAS-CHAVE

Palavras-chave: LangChain. Python. LLM.

EMENDAS



POSTECH